

Reviewer Pack — Minimal p-Adic Valuation Ring Complexity and DPLL Proof Path...

Entry c85d76c27c6e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 12:55:31 UTC

Minimal p-Adic Valuation Ring Complexity and DPLL Proof Path Length

Entry ID: c85d76c27c6e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 12:55:31 UTC

1. Conjecture

Field A (mathematical branch): p-adic Number Theory **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Path Length)

Statement:

For every instance φ of a Boolean satisfiability problem, the minimal number of p-adic valuations needed to express φ is in $\Theta(\log(n))$ of the DPLL proof path length for φ .

Rationale (proposer's reasoning):

p-adic numbers provide a framework that can capture the complexity of boolean functions by encoding their truth table as a sequence of valuations. If a direct correspondence exists between the number of valuations and the proof path length, it may reveal new insights into the nature of computational difficulty in satisfiability problems.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e71b0ace577d4b1b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for at least 25 out of 30 random seeds, the p-adic valuation ring complexity is within a factor of 2 from $\log(n)$ times the DPLL proof path length.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "p-adic valuation ring complexity" AND "DPLL proof path length" - "Boolean satisfiability problem" AND p-adic valuation - "minimal p-adic valuations" IN $\mathcal{O}(\log(n))$ DPLL

Top relevant hits considered: - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/2601.07595v3] Deep Search for Joint Sources of Gravitational Waves and High-Energy Neutrinos with IceCube During the Third Observing R

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import product

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def dpll(instance):
        if not instance:
            return 0
        for assignment in product([0, 1], repeat=len(instance[0])):
            if all(all(lit == (assignment[i] if lit >= 0 else -assignment[-lit-1]) for lit in clause) for clause in instance):
                return 1 + dpll([lit for lit in clause if lit != assignment[i]] for clause in instance)
        return float('inf')

    def p_adic_valuation_complexity(instance):
        valuations = set()
        for assignment in product([0, 1], repeat=len(instance[0])):
            valuation = sum(2**i if bit == 1 else 0 for i, bit in enumerate(assignment))
            valuations.add(valuation)
        return len(valuations)

    n_max = 40
    instances_tested = 30
    metric_values = []
    conjecture_holds = True
    counterexample = ""

    for n in range(5, n_max + 1):
        instance = [[random.randint(-n, n) for _ in range(random.randint(1, n))] for _ in range(n)]
        dpll_path_length = dpll(instance)
```

```

    if dpll_path_length == float('inf'):
        continue
    p_val_complexity = p_adic_valuation_complexity(instance)
    metric_values.append(p_val_complexity / math.log(n))

    if abs(p_val_complexity - math.log(n) * dpll_path_length) > 2 * math.log(n):
        conjecture_holds = False
        counterexample = f"Instance with n={n} failed. P-adic complexity: {p_val_complexity},

return {
    "metric_name": "P-adic Valuation Complexity / log(n)",
    "metric_value": sum(metric_values) / len(metric_values),
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']}\n first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5f17f9c1.py", line 67, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5f17f9c1.py", line 43, in run_trial
    dpll_path_length = dpll(instance)
                       ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5f17f9c1.py", line 24, in dpll
    if all(all(lit == (assignment[i] if lit >= 0 else -assignment[-lit-1]) for lit in clause) for clause
    ~~~~~

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5f17f9c1.py", line 24, in <genexpr>
    if all(all(lit == (assignment[i] if lit >= 0 else -assignment[-lit-1]) for lit in clause) for clause
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5f17f9c1.py", line 24, in <genexpr>
    if all(all(lit == (assignment[i] if lit >= 0 else -assignment[-lit-1]) for lit in clause) for clause
    ^
```

NameError: name 'i' is not defined. Did you mean: 'id'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a `NameError`, which prevented it from producing any data. As the critic challenged the conjecture and the support condition was not unambiguously met, the verdict is **INCONCLUSIVE**. | next: Debug the crashing issue in the test code and rerun the tests to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15211
2	propose	ollama_remote	glm4:latest	0	0	13070
3	preregistration	ollama_remote	glm4:latest	0	0	14016
4	novelty	ollama_remote	glm4:latest	0	0	8585
5	novelty	ollama_remote	glm4:latest	0	0	10259
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	62504
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9300
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11893
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10495
10	judge	ollama_remote	glm4:latest	0	0	12190

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 167523 ms total latency. Provider mix: {'ollama remote': 10}

(full prompt+response transcripts available in [research/audit/c85d76c27c6e.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c85d76c27c6e.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c85d76c27c6e.tar.gz (if generated)